



Disciplina: Projeto de Banco de Dados
Professor: Carlos Eduardo Cardoso

Especificação do Projeto Final

PokéManager API

1. Visão Geral do Sistema

A PokéManager API é uma aplicação backend moderna desenvolvida com o ecossistema Node.js, TypeScript e Computação em Nuvem. O objetivo do sistema é permitir que Treinadores Pokémon realizem cadastros, gerenciem suas coleções e montem seus times de até 6 Pokémons, integrando dados reais da PokéAPI para validação de estatísticas oficiais.

O projeto é continuativo e será construído do zero ao longo do semestre, aplicando princípios rigorosos de Clean Architecture, Padrões de Projeto, Qualidade de Software, Segurança e DevOps.

2. Stack Tecnológica

- **Linguagem & Runtime:** Node.js (v20+) + TypeScript (strict mode).
- **Framework HTTP:** Express ou Fastify.
- **Arquitetura:** Clean Architecture (Domain, Application, Infrastructure, Main).
- **Banco de Dados & ORM:** PostgreSQL + Prisma ORM (ou TypeORM / MongoDB Atlas).
- **Validação & Schemas:** Zod ou Joi.
- **Integração Externa:** Axios ou Fetch API para consumo da [PokéAPI](#).
- **Autenticação & Segurança:** JWT (JSON Web Token), bcrypt/argon2, Helmet, CORS, Rate Limiting.
- **Testes Automatizados:** Vitest ou Jest + Supertest (Mocks, Unitários e E2E).
- **DevOps & Cloud:** Docker, Docker Compose, GitHub Actions (CI/CD), Render/Fly.io/Railway.

3. Entregas do Projeto (4 Checkpoints)

Entrega 1: Arquitetura Limpa, Repositório In-Memory e Contrato REST

Prazo: 10/09 | **Peso:** 25% da Nota Final

Escopo: Criação da estrutura base com Clean Architecture e gerenciamento do catálogo local em memória.

Endpoints Mínimos:



- GET /api/v1/pokemons — Lista Pokémons do catálogo local (com suporte a req.query para filtros por tipo).
- GET /api/v1/pokemons/:id — Busca espécie local por ID (req.params).
- POST /api/v1/pokemons — Cadastro manual no catálogo local (req.body).
- PUT /api/v1/pokemons/:id — Atualização dos dados no catálogo local.
- DELETE /api/v1/pokemons/:id — Remoção do catálogo local.

Requisitos Técnicos:

- Configuração do tsconfig.json rigoroso com path aliases (@domain/*, @application/*, etc.).
- ESLint e Prettier integrados e sem alertas.
- Interface IPokemonRepository na camada de Domínio e implementação InMemoryPokemonRepository.
- Rota /api/docs fornecendo a documentação interativa com **Swagger / OpenAPI 3.0**.

Entrega 2: Persistência Relacional, Validação Zod e Integração PokéAPI

Prazo: 08/10 | **Peso:** 25% da Nota Final

Escopo: Conexão com Banco de Dados real e integração com a API pública oficial dos Pokémons.

Endpoints Mínimos:

- GET /api/v1/pokedex/search?name=pikachu — Consulta direto na PokéAPI externa e retorna o DTO formatado.
- POST /api/v1/trainers — Cadastro de novos Treinadores no banco de dados.
- POST /api/v1/trainers/:trainerId/captures — Captura de Pokémon:
 - Recebe o nome do Pokémon e o ID do treinador.
 - Consulta a PokéAPI oficial para buscar imagem, atributos base e tipos.
 - Valide a regra de negócio (Treinador não pode ter mais de 6 Pokémons no time ativo).
 - Salva a captura no Banco de Dados associada ao treinador.
- GET /api/v1/trainers/:trainerId/team — Lista o time atual do treinador.

Requisitos Técnicos:

- Implementação do PrismaPokemonRepository substituindo o InMemory sem alterar a camada de Domínio.
- Criação da interface IPokeApiGateway e implementação com Axios/Fetch em infrastructure/providers/.
- Validação de payload de todas as requisições utilizando Zod.
- Middleware de Tratamento Global de Erros (AppError) e exceções customizadas do domínio.



Entrega 3: Autenticação JWT, Autorização RBAC e Hardening OWASP

Prazo: 05/11 | **Peso:** 25% da Nota Final

Escopo: Módulo de segurança completo e proteção contra vulnerabilidades web.

Endpoints Mínimos:

- POST /api/v1/auth/register — Cadastro de treinador com senha criptografada.
- POST /api/v1/auth/login — Autenticação e emissão de **JWT Bearer Token** e Refresh Token.
- POST /api/v1/me/team/catch — Rota protegida (utiliza o ID do treinador extraído do Token JWT).
- DELETE /api/v1/me/team/:captureId — Soltar Pokémon (apenas o próprio dono autenticado pode soltar).
- PATCH /api/v1/trainers/:id/role — Alteração de perfil de acesso (Protegida por **RBAC**: Apenas papel GYM_LEADER/Admin).

Requisitos Técnicos:

- Criptografia de senhas com bcrypt ou argon2.
- Middleware de autenticação JWT validando cabeçalho Authorization: Bearer <token>.
- Aplicação de middlewares de segurança: **Helmet** (HTTP Headers), **Cors** e **Rate Limiter** (Prevenção contra Brute Force).
- DTOs e Serialização para garantir que **hash de senhas e dados sensíveis nunca vazem no JSON**.
- Validação de .env usando Zod no startup do servidor.

Entrega 4: Docker, Testes, CI/CD e Cloud Deploy

Prazo: 03/12 | **Peso:** 25% da Nota Final

Escopo: Automação, containerização, suíte de testes e deploy da aplicação em produção.

Entregáveis Obrigatórios:

- **Docker & Compose:**
 - Dockerfile otimizado em múltiplos estágios (*multi-stage build*).
 - docker-compose.yml capaz de subir a API + Banco de Dados PostgreSQL com o comando docker compose up.
- **Suíte de Testes (Vitest ou Jest):**
 - **Testes Unitários:** Testes isolados dos Casos de Uso (ex: validação do limite de 6 Pokémons) utilizando Repositórios In-Memory.
 - **Testes de Integração/E2E (Supertest):** Testes das rotas HTTP principais (Register, Login, Catch, List).



Integração Contínua (CI):

- Pipeline `.github/workflows/ci.yml` configurada no GitHub para rodar Linters, Typecheck e Testes a cada Push/PR.

Deploy Cloud & Produção:

- Publicação da API em serviço de hospedagem cloud (Render, Fly.io ou Railway).
- Banco de dados hospedado em nuvem (Neon, Supabase ou Mongo Atlas).
- Documentação Swagger funcional e ativa na URL de produção.

4. Sugestão do README.md Final

```
# 🎮 PokéManager API

![CI Workflow](https://img.shields.io/github/actions/workflow/status/seu-usuario/pokemanager-api/ci.yml)
![License](https://img.shields.io/badge/license-MIT-blue)
![TypeScript](https://img.shields.io/badge/TypeScript-5.0-blue)

API RESTful para gerenciamento de Treinadores e Pokémons desenvolvida na disciplina de
**Desenvolvimento de APIs Modernas**.

🔗 **URL de Produção:** [https://pokemanager-api.onrender.com](https://pokemanager-api.onrender.com)

📖 **Documentação Swagger:** [https://pokemanager-api.onrender.com/api/docs](https://pokemanager-api.onrender.com/api/docs)

## 🛠️ Tecnologias Utilizadas
- Node.js & TypeScript
- Express / Clean Architecture
- Prisma ORM & PostgreSQL
- Docker & Docker Compose
- Vitest & Supertest
- GitHub Actions (CI/CD)

## 🚀 Como Executar o Projeto Localmente

### Pré-requisitos
- Docker e Docker Compose instalados.
```



```
``bash
# 1. Clone o repositório
$ git clone [https://github.com/seu-usuario/pokemanager-
api.git](https://github.com/seu-usuario/pokemanager-api.git)

# 2. Acesse a pasta
$ cd pokemanager-api

# 3. Configure as variáveis de ambiente
$ cp .env.example .env

# 4. Suba a aplicação via Docker
$ docker compose up -d

# 5. Execute os testes
$ npm run test
```